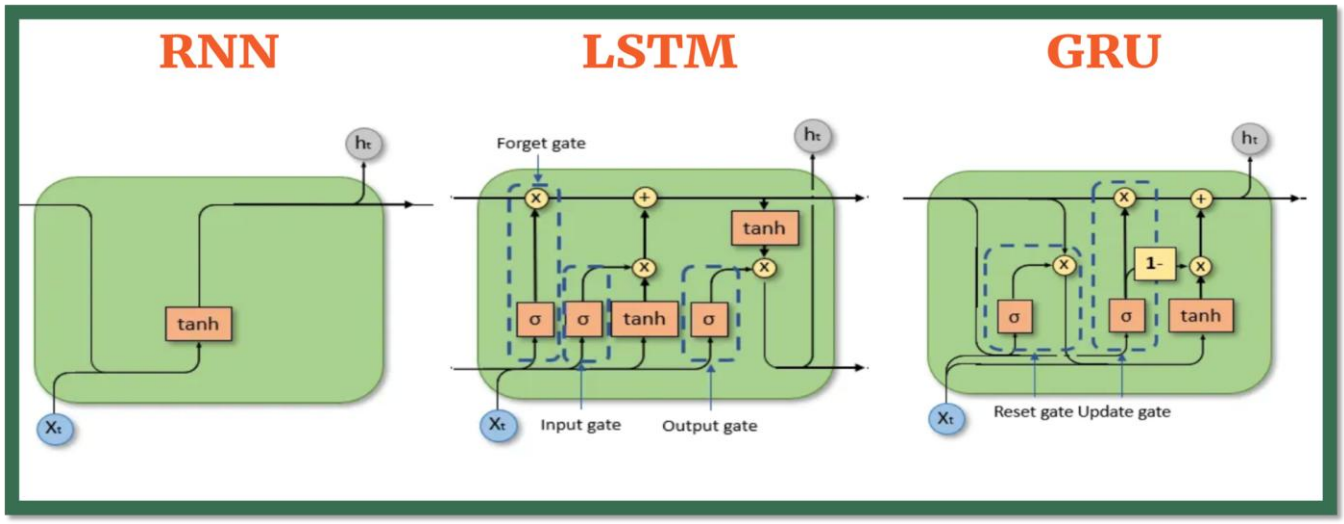
Feedback Networks (RNN, LSTM, GRU)

Advantage: These models excel at processing sequential data by maintaining an internal **hidden state** (memory) that allows them to model temporal dependencies, making them highly effective for tasks like time series prediction and basic language modeling. LSTMs and GRUs specifically solved the **vanishing gradient problem** that plagued traditional RNNs, enabling them to capture very long-range dependencies.

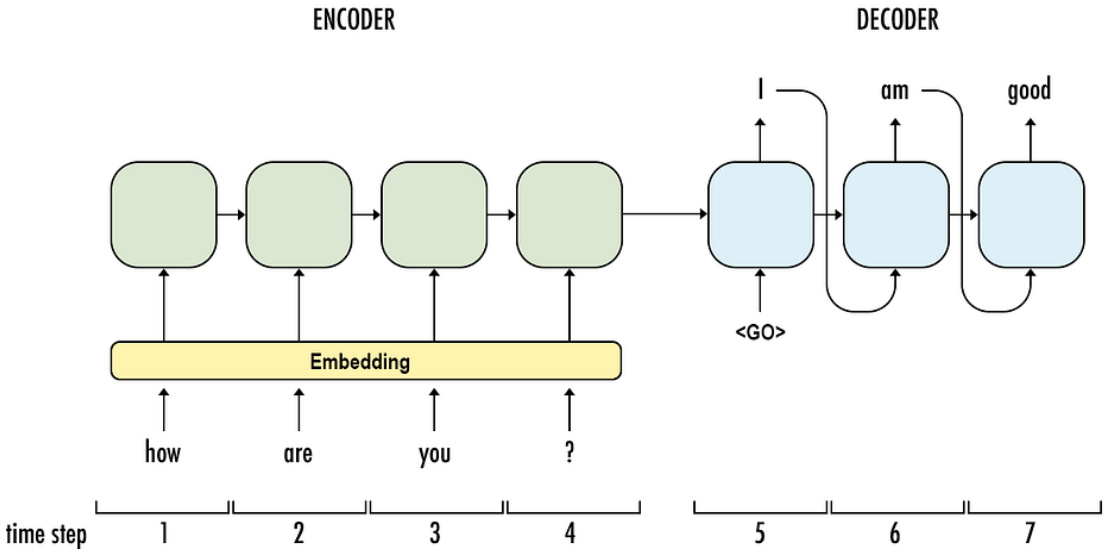
Limitation: Their architecture is inherently **sequential**, meaning they must process data one token at a time. This prevents **parallel computation** during training, making them significantly slower and computationally inefficient compared to feed-forward models when handling very long sequences.



Sequence-to-Sequence Models (Encoder-Decoder)

Advantage: The architecture cleanly separates the concerns of **understanding** (encoding the source sequence into a fixed-length context vector) and **generation** (decoding the target sequence from that vector). This makes them the standard framework for tasks where the input and output lengths differ, such as **Machine Translation** and summarization.

Limitation: The original architecture forces the entire source sequence's meaning to be compressed into a single **fixed-length context vector** (the bottleneck). This bottleneck often struggles to retain all the necessary information for accurately decoding very long or complex input sequences.

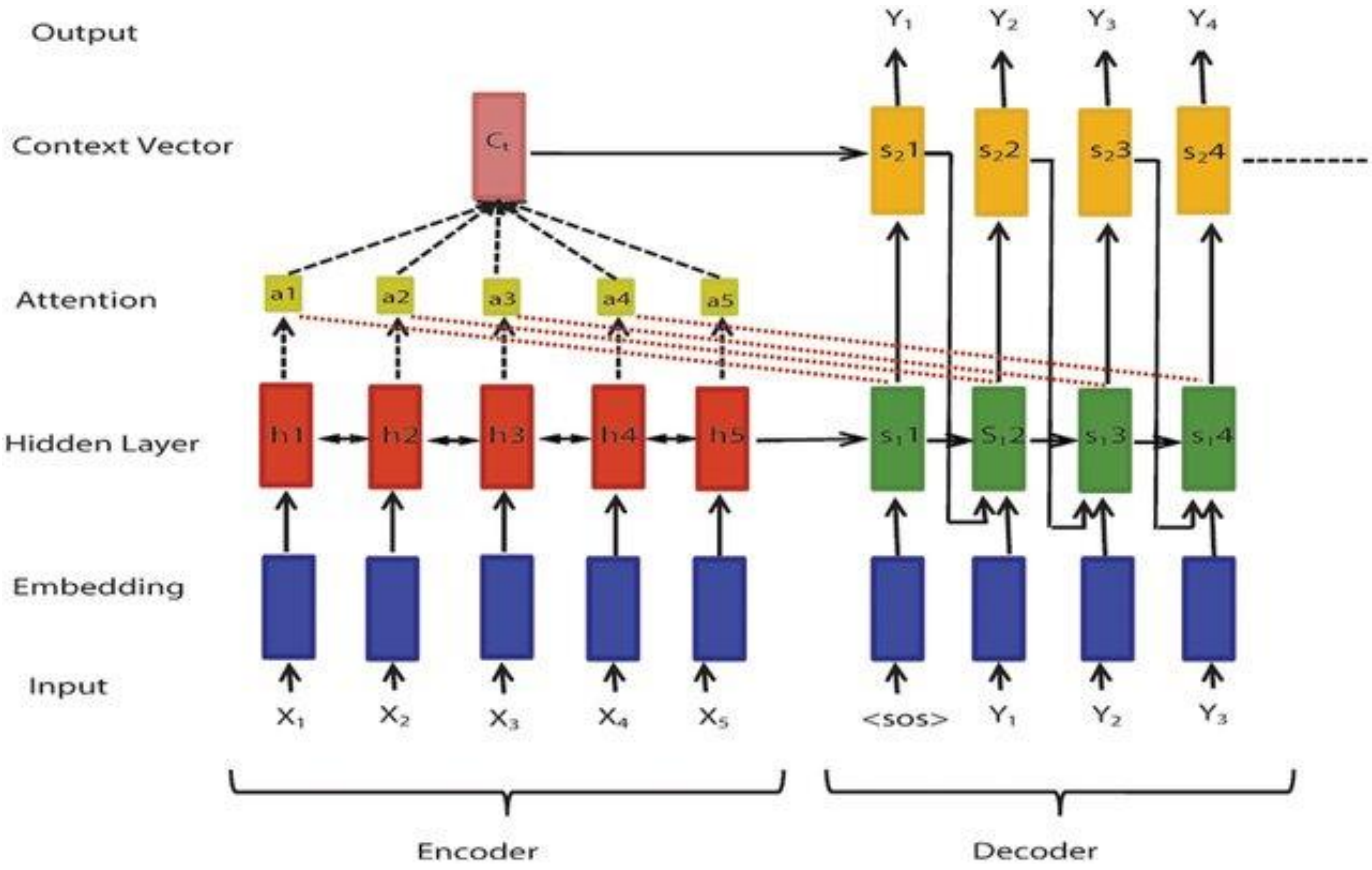


Attention Mechanism

Advantage: Solves the fixed-length bottleneck of the original Seq2Seq model by generating a **dynamic context vector** for every word. This allows the decoder to look back at the entire source sentence and **selectively focus** on the most relevant parts, drastically improving translation quality, especially for long sentences.

Limitation: While better than pure Seq2Seq, the attention mechanism, particularly the **Additive or Dot-Product Attention** used in earlier NMT models, still requires calculating attention scores between the current decoder state and *every* single encoder hidden state. This remains computationally intensive for **extremely long sequences**, leading to slower training times compared to the later, parallelized Transformer architecture.

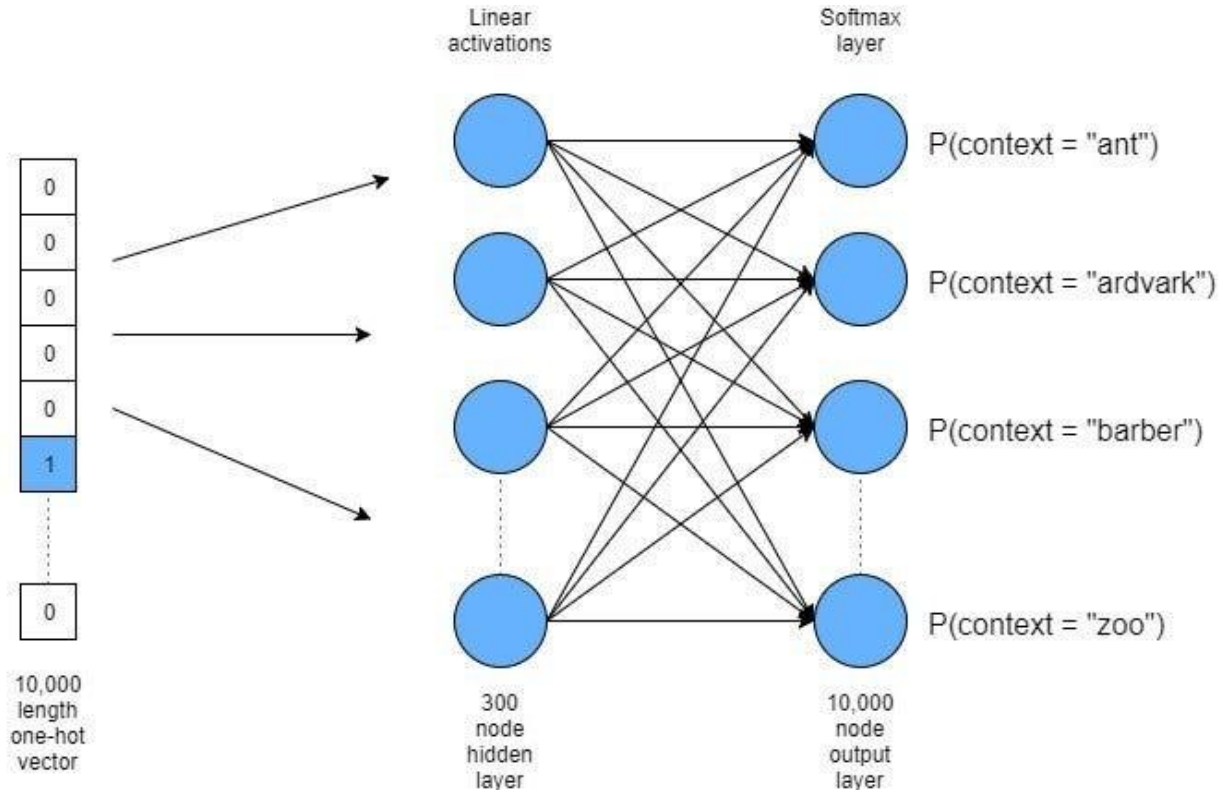
In the sentence, "The **bank** near the river is closed, but the **bank** said funds are available," the final vector for the first "bank" will be contextually related to "river" and "closed," while the vector for the second "bank" will be related to "funds" and "available."

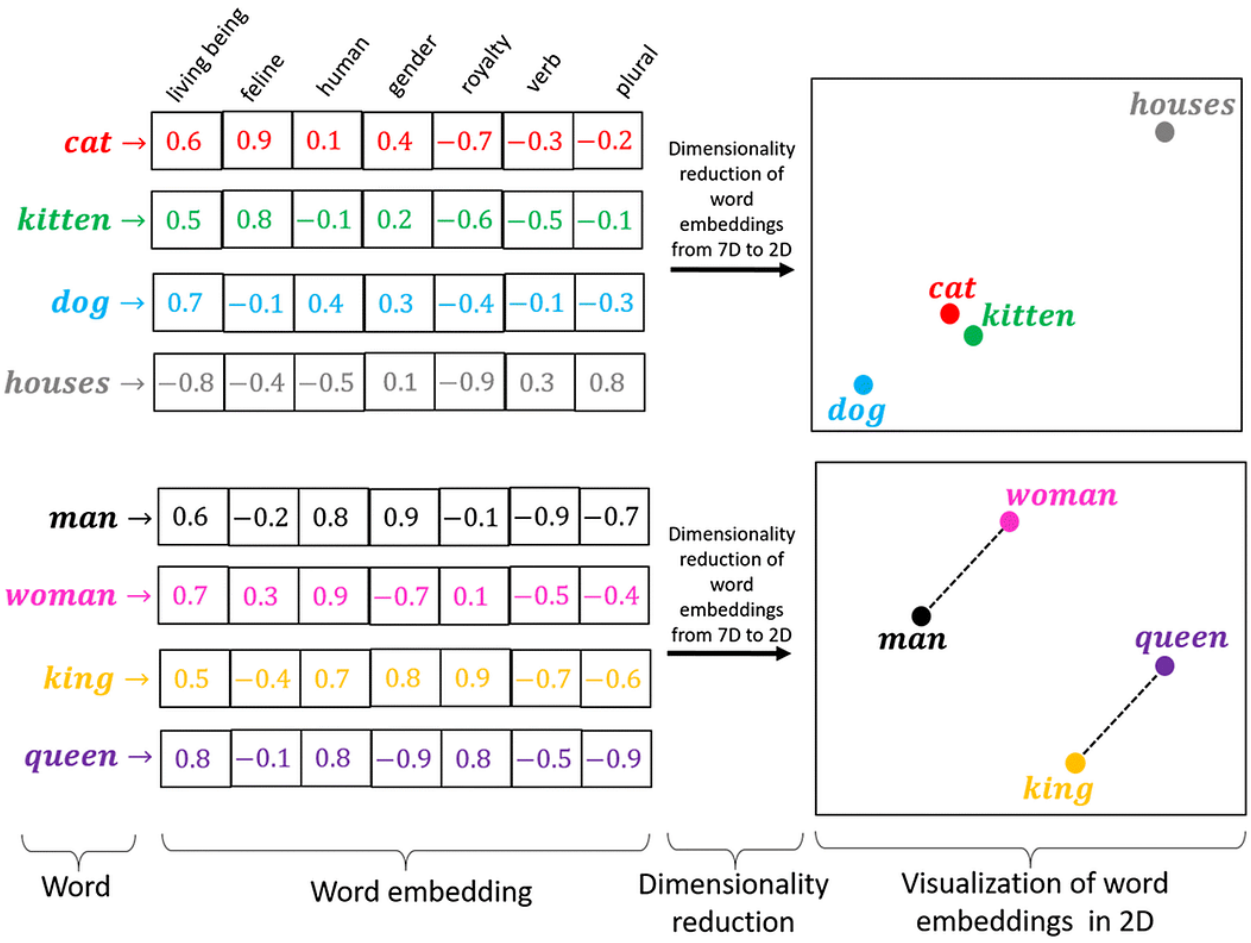


Word Embeddings

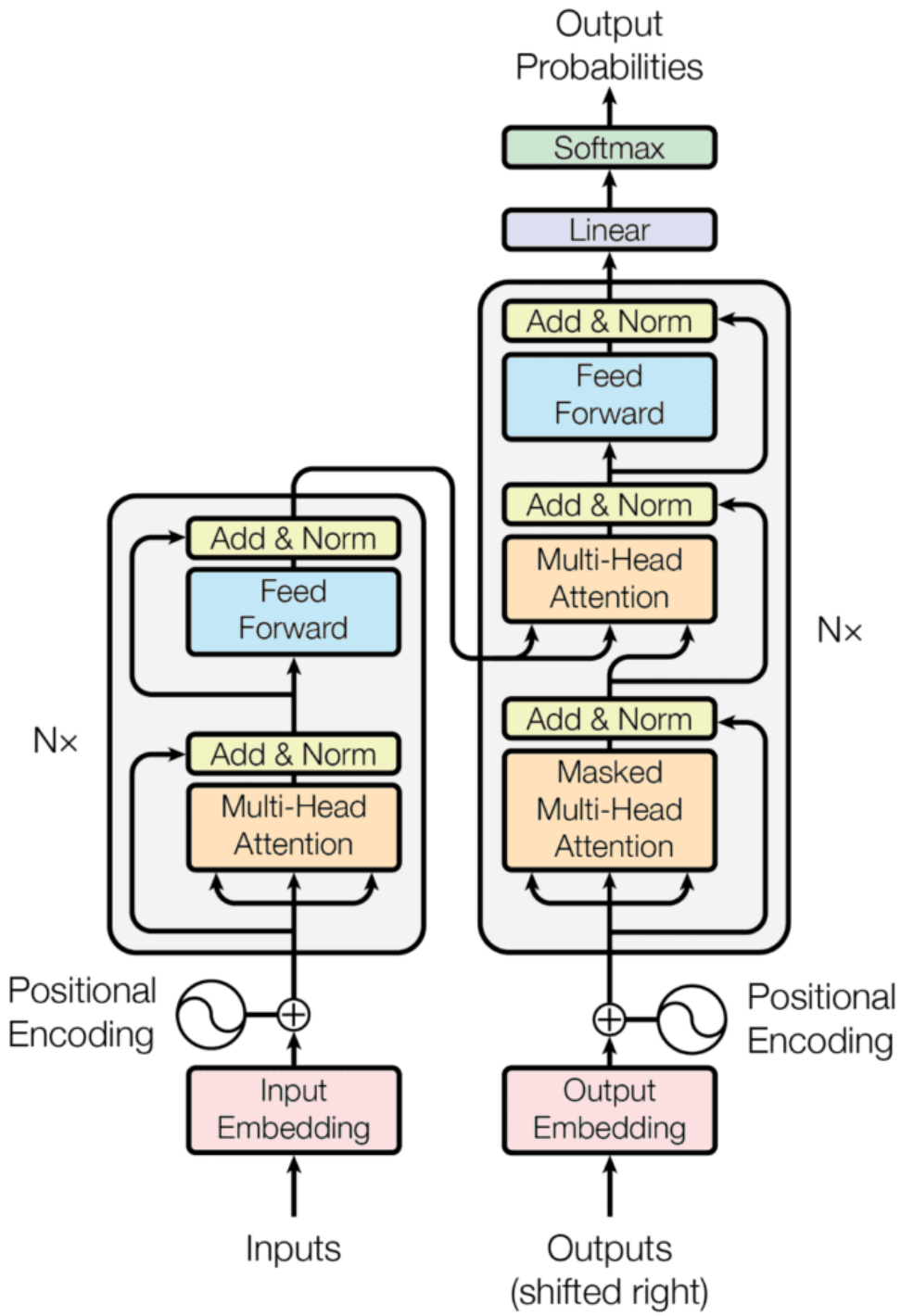
Word embeddings are **dense vector representations** of words that numerically capture their semantic meaning and contextual usage. The fundamental idea is that words used in similar contexts are mapped close together in the vector space, allowing mathematical operations (like vector addition) to model linguistic relationships (e.g., gender or tense). They serve as the critical input layer for nearly all modern NLP models, moving beyond simple symbolic representations.

The main limitation of **static word embeddings** (like Word2Vec) is their inability to handle **polysemy** (multiple meanings) and **contextual ambiguity**. For example, the word **"Apple"** receives a single, fixed vector whether it refers to the **fruit** ("I ate a red apple") or the **technology company** ("I bought a new Apple iPhone"), failing to capture the entirely different meanings of the word based on its context. This lack of dynamic representation limits their effectiveness compared to newer contextual models.




living being
feline
human
gender
royalty
verb
plural
cat → 0.6 0.9 0.1 0.4 −0.7 −0.3 −0.2
kitten → 0.5 0.8 −0.1 0.2 −0.6 −0.5 −0.1
dog → 0.7 −0.1 0.4 0.3 −0.4 −0.1 −0.3
houses → −0.8 −0.4 −0.5 0.1 −0.9 0.3 0.8
Dimensionality reduction of word embeddings from 7D to 2D
houses
cat
kitten
dog
man → 0.6 −0.2 0.8 0.9 −0.1 −0.9 −0.7
woman → 0.7 0.3 0.9 −0.7 0.1 −0.5 −0.4
king → 0.5 −0.4 0.7 0.8 0.9 −0.7 −0.6
queen → 0.8 −0.1 0.8 −0.9 0.8 −0.5 −0.9
Dimensionality reduction of word embeddings from 7D to 2D
woman
man
queen
king
Word
Word embedding
Dimensionality reduction
Visualization of word embeddings in 2D

Transformer (Attention is all you need)



Positional Embedings

The core of the Transformer—the self-attention mechanism—processes all tokens in a sequence **simultaneously and in parallel**. Because there is no recurrence (like in an RNN) or sequential processing, the model inherently loses all information about the original word order. Without positional information, the model would get the same result for the sentence "Dog bites man" and "Man bites dog."

A separate vector, PE, is generated for every position t in the sequence. These vectors are not learned from the data in the original Transformer paper; they are calculated using fixed, pre-defined mathematical functions (sine and cosine waves). This allows the model to generalize to sequences longer than those seen during training.

The positional embedding vector PE is simply **added** to the content embedding vector E for the word at position t.

$$PE_{(pos,2i)} = sin(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

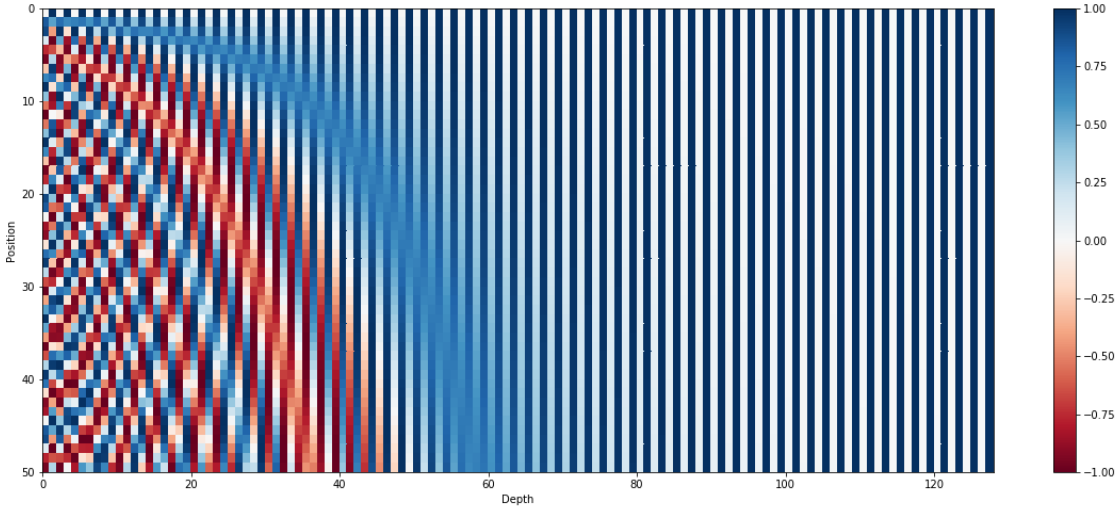
$$PE_{(pos,2i+1)} = cos(\frac{pos}{10000^{2i/d_{\text{model}}}})$$

Example for pos 0:

$$PE_{(0,0)} = \sin\left(\frac{0}{\text{Denominator}}\right) = \sin(0) = \mathbf{0} \qquad PE_{(0,1)} = \cos\left(\frac{0}{\text{Denominator}}\right) = \cos(0) = \mathbf{1}$$

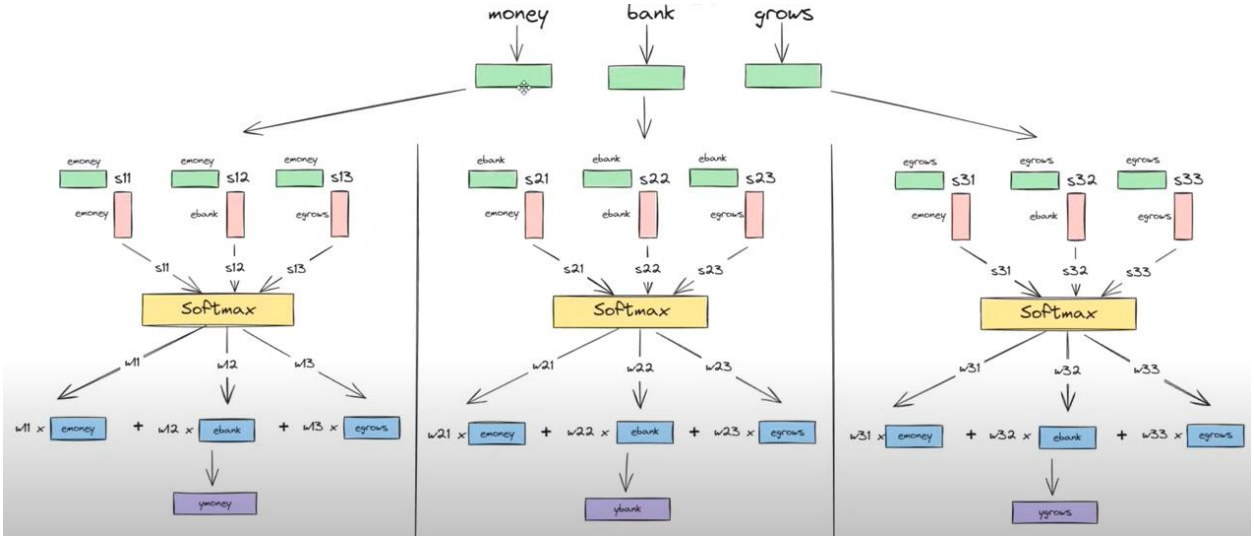
$$PE_{(0,2)} = \sin\left(\frac{0}{\text{Denominator}}\right) = \sin(0) = \mathbf{0} \qquad PE_{(0,3)} = \cos\left(\frac{0}{\text{Denominator}}\right) = \cos(0) = \mathbf{1}$$

$$PE_{\mathbf{pos}=\mathbf{0}} = [\mathbf{0}, \mathbf{1}, \mathbf{0}, \mathbf{1}]$$

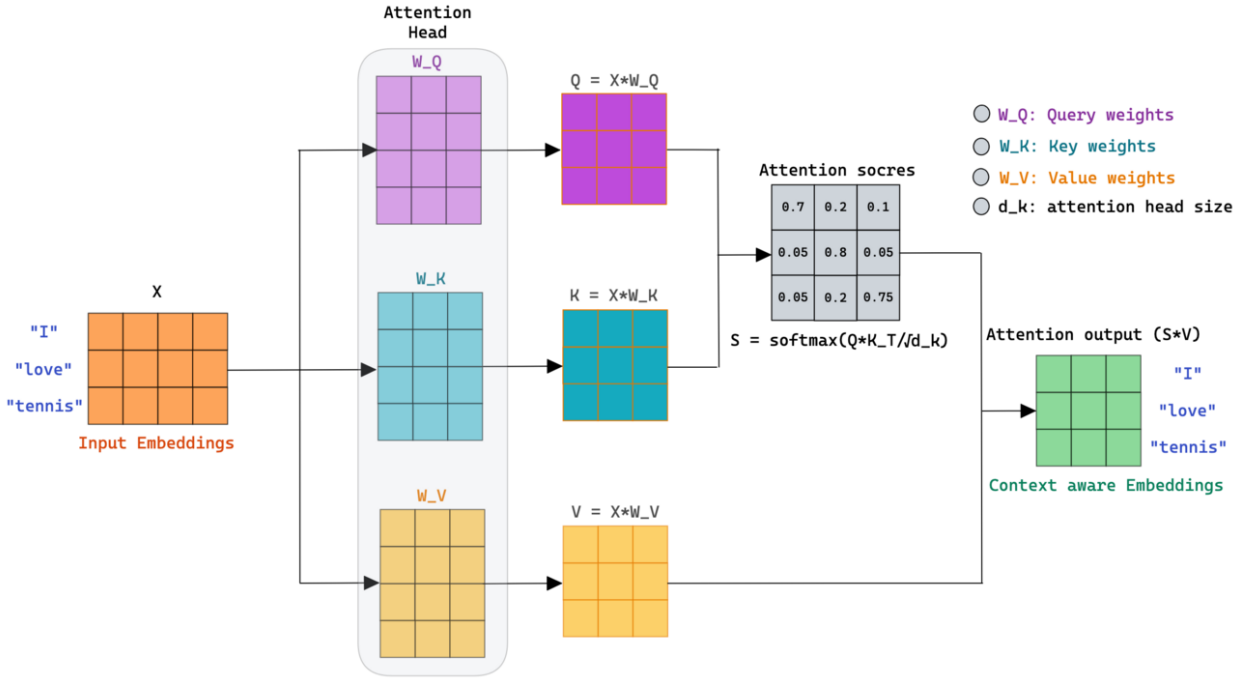


Self Attention

Instead of processing data sequentially (like an RNN) or locally (like a CNN), self-attention enables every token in a sentence (or every patch in an image) to directly **attend to** and **exchange information** with every other token in that same sequence.



https://www.youtube.com/watch?v=-tCKPl_8Xb8



Word	Static Embedding (4-d)
He	[1, 0, 1, 0]
eats	[0, 1, 0, 1]
apples	[1, 1, 0, 0]

$$W_Q = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}, \quad W_V = I_{4\times 4}$$

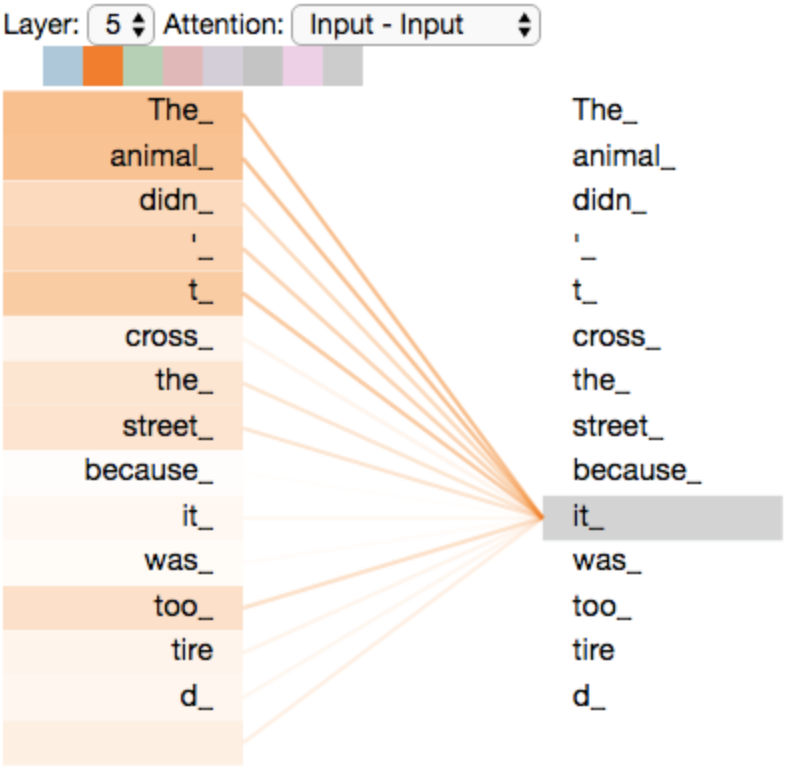
$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V = X$$

Q	K	V
$\begin{bmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$	$\begin{bmatrix} 2 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 2 & 0 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 \end{bmatrix}$

$$QK^T = \begin{bmatrix} 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 4 & 5 \\ 3 & 5 & 5 \\ 4 & 4 & 4 \end{bmatrix}$$

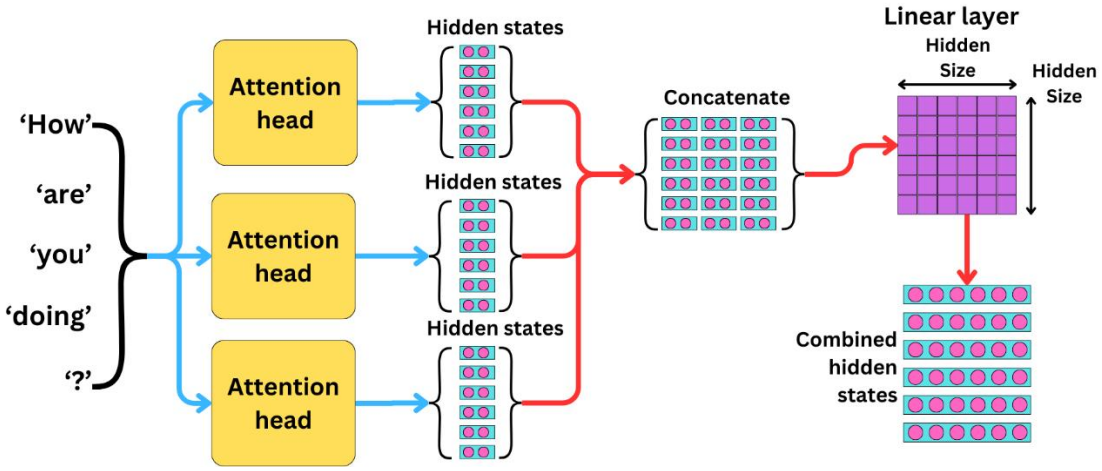
$$\text{Scaled Scores} = \begin{bmatrix} 3 & 2 & 2.5 \\ 1.5 & 2.5 & 2.5 \\ 2 & 2 & 2 \end{bmatrix} \quad \alpha = \begin{bmatrix} 0.51 & 0.19 & 0.31 \\ 0.16 & 0.42 & 0.42 \\ 0.33 & 0.33 & 0.33 \end{bmatrix}$$

Word	Contextual embedding
He	[0.82, 0.50, 0.51, 0.19]
eats	[0.58, 0.84, 0.16, 0.42]
apples	[0.66, 0.66, 0.33, 0.33]


Layer: 5
Attention: Input - Input
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_
The_
animal_
didn_
'_
t_
cross_
the_
street_
because_
it_
was_
too_
tire
d_

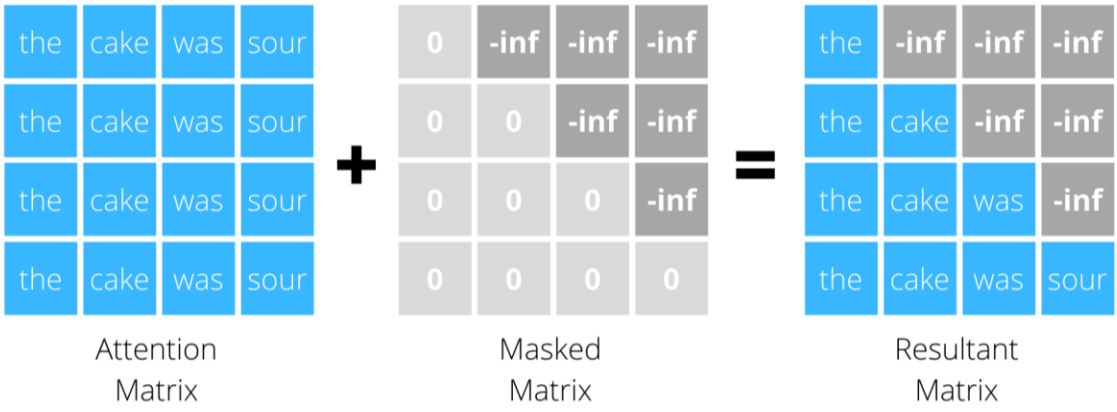
Multi-head Attention

Multi-head attention enhances the self-attention mechanism by performing the attention calculation **multiple times in parallel** using different, independently learned sets of **Query, Key, and Value** projection matrices. This allows the model to simultaneously focus on **different types of relationships** or **different parts of the inputs.**



Masked Attention

Masked Attention, specifically used in the **Decoder** of the Transformer architecture, is a modification that prevents a token from attending to **subsequent (future) tokens** in the sequence. It achieves this by setting the attention scores for future positions to negative infinity before the Softmax function, essentially forcing those weights to zero. This ensures the model maintains the **autoregressive** property during training, meaning it predicts the next word based only on the words it has already "seen" (generated).



Word	Static Embedding (4-d)
He	[1, 0, 1, 0]
eats	[0, 1, 0, 1]
apples	[1, 1, 0, 0]

Head#1

$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$	$W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$	$W_V =$ $\mathbf{I_{4x2}}$
Head#2		
$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$	$W_K = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}$	$W_V =$ $\mathbf{I_{4x2}}$

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V = X$$

Q	K	V
(3x4).(4x2) = (3x2)	**(3x4).(4x2) = (3x2)**	**(3x4).(4x2) = (3x2)**

QK^T = **(3x2).(2x3) = (3x3)**

Output = (3x3).(3x2) = (3x2)

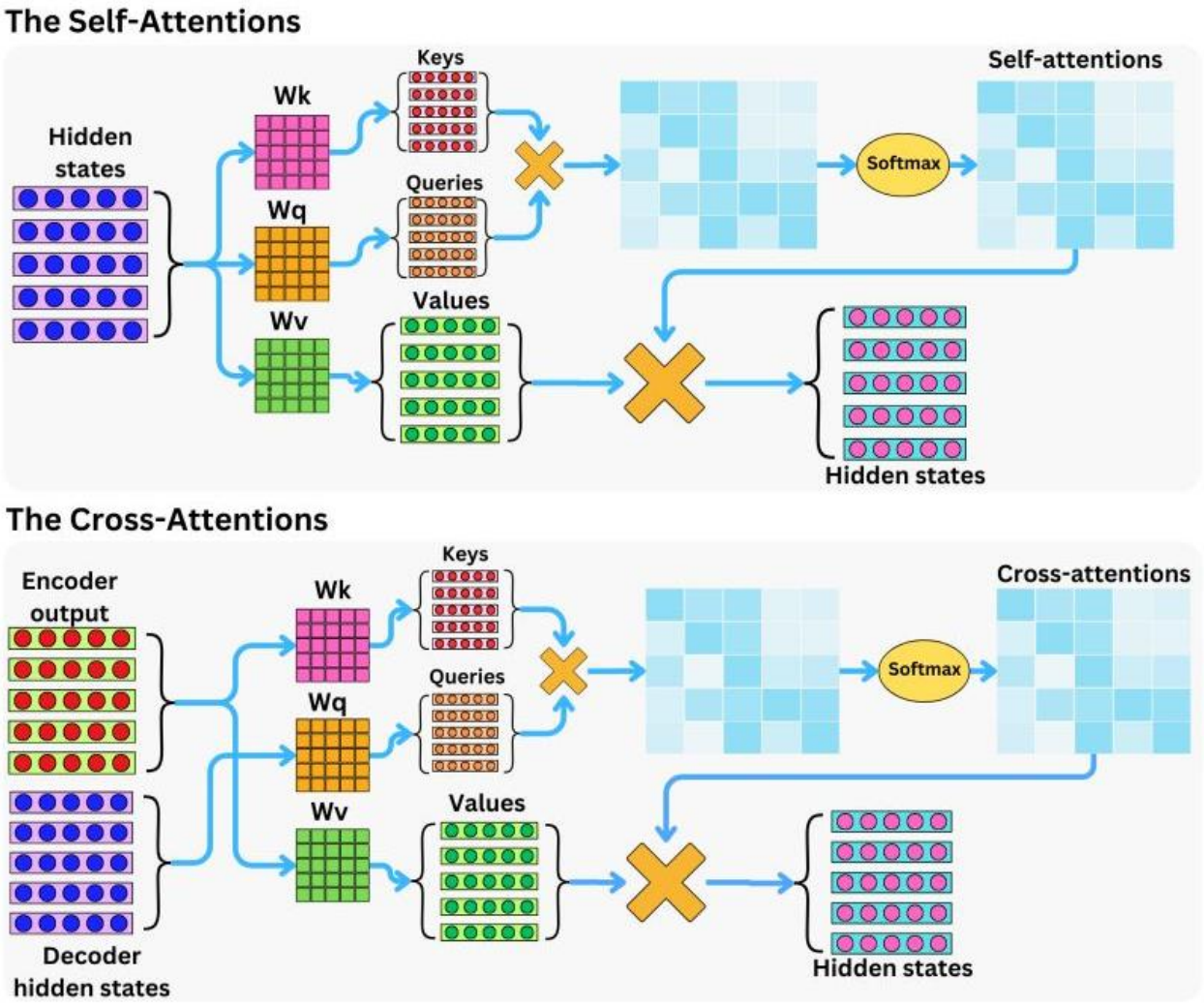
Head 1 Output = 3x2, Head 2 Output = 3x2

Concat = (3x4)

Word	Contextual embedding
He	[0.82, 0.50, 0.51, 0.19]
eats	[0.58, 0.84, 0.16, 0.42]
apples	[0.66, 0.66, 0.33, 0.33]

Cross Attention (Encoder-Decoder Attention)

Cross-attention operates similarly to self-attention but uses vectors from two different input sequences rather than one. The cross-attention mechanism calculates the relevance (attention scores) between the Decoder's current state Q and every token in the Encoder's output K. It then creates a weighted sum of the source content V based on those scores. This allows the Decoder to dynamically align parts of the target output (e.g., a French word being generated) with the corresponding, relevant parts of the source input (e.g., the English phrase). This solves the fixed-length bottleneck by giving the decoder a direct, dynamic view of the entire source context at every step.



Vision Transformer (ViT)

The Vision Transformer (ViT) is an encoder-only model that adapts the successful **Transformer architecture** from NLP for image classification. It works by dividing an input image into a sequence of small, non-overlapping **patches** which are treated as tokens. These tokens are then processed by the self-attention mechanism, allowing the model to capture **global relationships** between all parts of the image simultaneously, bypassing traditional convolution entirely.

The Vision Transformer (ViT) does not have a decoder because its primary task is **image classification**, which is a non-generative task focused on **understanding** (encoding) the image content, not creating a new output sequence.

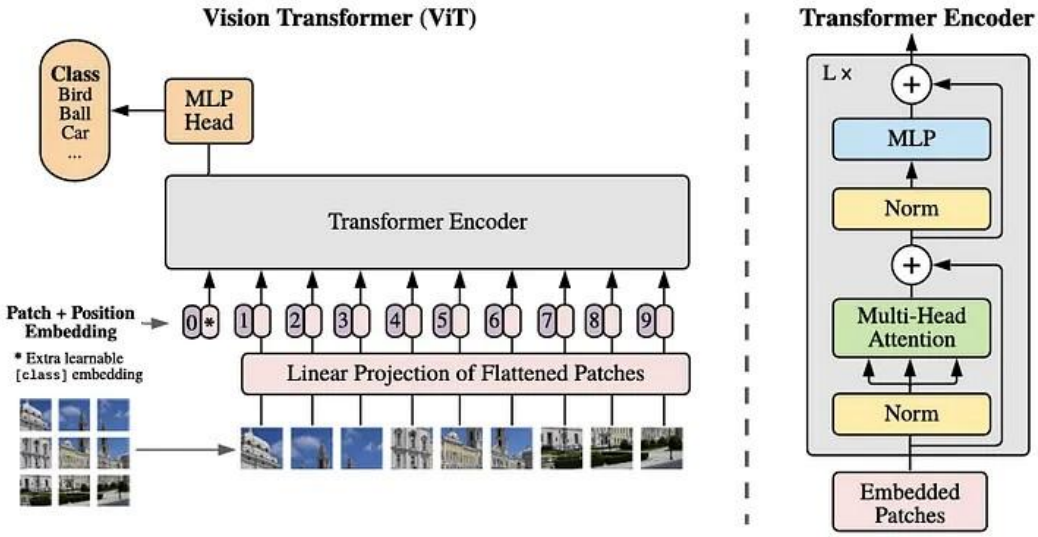


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

1. Image Patch Embedding (Tokenization)

This step converts the 2D image into a sequence of linear, learnable vectors, similar to how a sentence is broken into word tokens.

Example Setup

- **Input Image:** A 16×16 pixel color image (e.g., $16 \times 16 \times 3$).
- **Patch Size:** 4×4.
- **Target Embedding Dimension ($\mathbf{D}$):** 8.

The Process

1. **Patching:** The image is divided into non-overlapping patches.
   - Number of patches: $\frac{16 \times 16}{4 \times 4} = 4 \times 4 = 16$ patches.
2. **Flattening:** Each patch is flattened into a 1D vector.
   - Patch vector dimension: $4 \times 4 \times 3 \ (\text{RGB}) = 48$.
3. **Linear Projection (Embedding):** A **learnable linear layer** ($\mathbf{W}_{\text{embed}}$) is applied to each flattened patch vector to project it into the target embedding dimension ($\mathbf{D} = 8$).

$$\text{Patch Embedding } = \text{Flattened Patch} \times \mathbf{W}_{\text{embed}}$$

   - $\text{Shape} : (1 \times 48) \times (48 \times 8) = (1 \times 8)$

The output of this step is a sequence of 16 embedded vectors, $\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_{16}$, each with a dimension of 8.

2. Class Token and Sequence Assembly

To perform classification, an extra, learnable **Class Token** ($\mathbf{E}_{\text{CLS}}$) is added to the beginning of the sequence.

- This token is treated like any other patch and its final output vector (after passing through the Transformer layers) is used as the **final image representation** for the classification head.
- The final sequence length is $16 \ (\text{patches}) + 1 \ (\text{CLS}) = 17$ vectors, each of dimension 8.

3. Positional Embedding (PE)

Since the Transformer's self-attention mechanism processes all tokens in the sequence simultaneously (it is **permutation-invariant**), it has no intrinsic knowledge of where a patch originated (e.g., top-left vs. bottom-right).

The Process

1. **Positional Vectors:** A set of **learnable Positional Embedding vectors** ($\mathbf{P}_0, \mathbf{P}_1, \ldots, \mathbf{P}_{16}$) are created, one for each position (including the CLS token). These vectors are trained alongside the network.
    - Each $\mathbf{P}_i$ has the same dimension as the patch embeddings ($\mathbf{D} = 8$).
2. **Addition:** The positional vector is simply **added element-wise** to its corresponding patch embedding.

$$\mathbf{Z}_i = \mathbf{E}_i + \mathbf{P}_i$$

Example

Token	Initial Embedding ($\mathbf{E}_i$, 1×8)	Positional Embedding ($\mathbf{P}_i$, 1×8)	Final Input Sequence ($\mathbf{Z}_i$)
CLS	$\mathbf{E}_{\text{CLS}}$ (Learnable)	$\mathbf{P}_0$ (Learnable)	$\mathbf{Z}_0 = \mathbf{E}_{\text{CLS}} + \mathbf{P}_0$
Patch 1	$\mathbf{E}_1$ (From top-left patch)	$\mathbf{P}_1$ (Learnable)	$\mathbf{Z}_1 = \mathbf{E}_1 + \mathbf{P}_1$
Patch 16	$\mathbf{E}_{16}$ (From bottom-right patch)	$\mathbf{P}_{16}$ (Learnable)	$\mathbf{Z}_{16} = \mathbf{E}_{16} + \mathbf{P}_{16}$

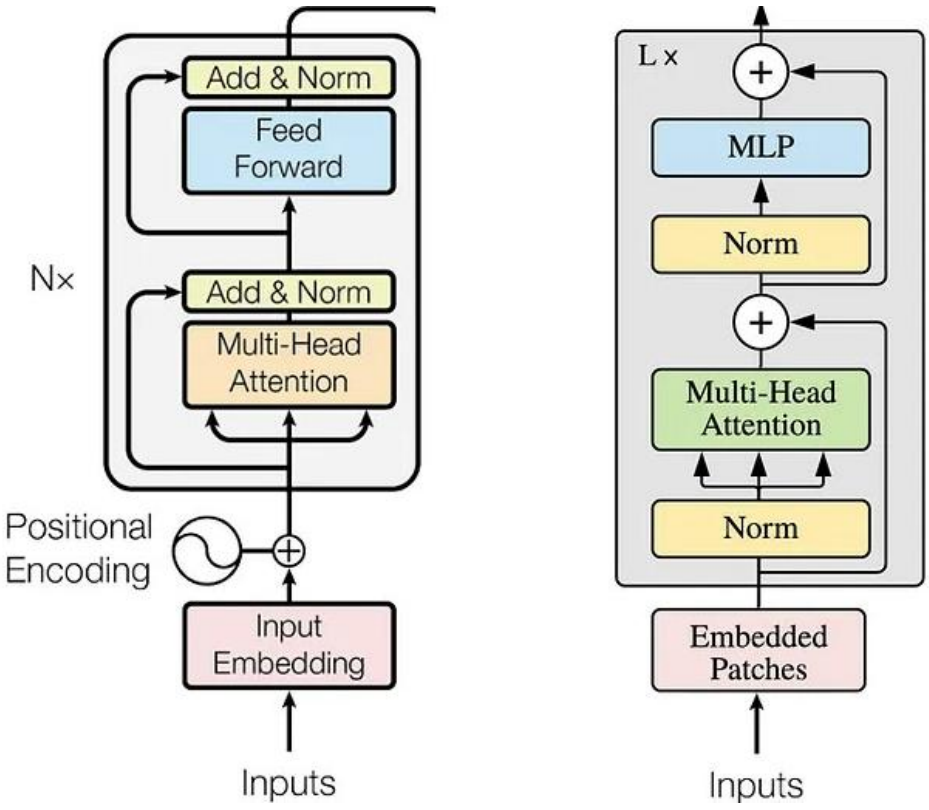
ViT Output Layer

The ViT's output layer is a simple **Multi-Layer Perceptron (MLP) classification head** that operates only on the output vector corresponding to the special **[CLS] token**.

1. **Input:** After the entire sequence of tokens (patches + CLS token) passes through all the Transformer Encoder blocks, the output is a set of L vectors, $Z_0, Z_1, \ldots, Z_{L-1}$. The model only uses Z_0, the vector corresponding to the initial **[CLS] token**.
2. **Function:** The Z_0 vector is considered the **aggregated, final representation of the entire image**. It is passed to a small **MLP head** (often with one hidden layer and a non-linear activation like ReLU).
3. **Output:** The final layer of the MLP head uses the **Softmax** activation function (for classification) to produce a probability distribution over the target classes.

Vit Output layer Vs CNN Output Layer

Feature	Vision Transformer (ViT)	Traditional CNN (e.g., VGG, ResNet)
Source of Features	A single, **synthesized vector** (**[CLS] token**). This vector has been fully contextualized by **self-attention** from all other image patches.	A spatially structured **Feature Map** (e.g., $7 \times 7 \times 512$). This map retains some spatial grid information.
Transition Layer	A simple **Linear Projection/MLP Head** that takes the $1 \times D$ vector.	Typically, a **Global Average Pooling (GAP)** layer is first applied to convert the 3D feature map (e.g., $7 \times 7 \times 512$) into a 1×512 vector.
Final Classifier	A **Linear/MLP Layer** followed by Softmax.	A **Fully Connected (FC) Layer** followed by Softmax.


Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Input Embedding
Inputs
L x
MLP
Norm
Multi-Head Attention
Norm
Embedded Patches
Inputs

